

AI Learning Roadmap — Java Backend → GenAI/Agentic AI Engineer

Built for: Java Full Stack (Spring Boot) background, with prior hands-on GenAI (Anthropic API expense tracker). How to use: Search each topic as-is. Spend 1-3 days per topic depending on depth needed. Build something small after every phase — don't just read.

Phase 0: Python Basics for AI (1 week)

Most AI tooling is Python-first, even though you'll deploy in Java. You need just enough to read/prototype.

- Python syntax for Java developers (crash course)
- Python virtual environments (venv/conda)
- NumPy basics
- Pandas basics for data handling
- Python async/await (asyncio) basics
- Jupyter notebooks basics

Checkpoint: Can read and modify a basic Python AI script without getting stuck on syntax.

Phase 1: LLM & GenAI Fundamentals (1-2 weeks)

Understand what you're actually working with before building on top of it.

- What is a large language model (LLM) explained simply
- Transformer architecture explained
- Tokens and tokenization explained
- Context window explained
- Temperature, top-p, top-k in LLMs
- System prompt vs user prompt
- Prompt engineering techniques
- Few-shot prompting vs zero-shot prompting
- Chain-of-thought prompting
- LLM hallucination explained
- Anthropic API documentation (you've used this already — go deeper)
- OpenAI API vs Anthropic API vs open-source LLMs comparison
- Function calling / tool use in LLMs
- Structured output from LLMs (JSON mode)

Checkpoint: Can explain how an LLM call works end-to-end, and can design a good system prompt.

Phase 2: RAG (Retrieval-Augmented Generation) (2-3 weeks)

This is the highest-ROI skill area right now and connects directly to your JPA/data background.

- What is RAG (Retrieval-Augmented Generation) explained
- Embeddings explained (text embeddings)
- Vector databases explained
- Pinecone vs Chroma vs Weaviate vs pgvector comparison
- pgvector with PostgreSQL tutorial (natural fit — you already know Postgres/MySQL)
- Chunking strategies for RAG
- Semantic search vs keyword search
- Cosine similarity explained
- RAG pipeline architecture
- Reranking in RAG systems
- Hybrid search (keyword + vector) explained
- Evaluating RAG systems (RAGAS, evaluation metrics)
- Common RAG failure modes and fixes

Checkpoint: Build a basic RAG app — feed it a PDF/docs, ask questions, get grounded answers.

Phase 3: Java-Native AI Integration (2-3 weeks)

Bring it into your actual stack — this is your differentiator.

- LangChain4j tutorial
- Spring AI tutorial
- Spring AI RAG implementation
- Spring AI with pgvector
- LangChain4j with Spring Boot integration
- Calling Anthropic Claude API from Java
- Streaming responses from LLM in Spring Boot (SSE/WebFlux)
- Building a chatbot backend with Spring Boot and LLM
- Rate limiting and retry strategies for LLM API calls
- Caching LLM responses (cost optimization)

Checkpoint: Rebuild your expense tracker (or a new small app) using Spring AI or LangChain4j instead of raw API calls — add a RAG component to it.

Phase 4: Agentic AI (2-3 weeks)

The next wave, and where premium pay is forming.

- What is agentic AI explained
- AI agents vs chatbots explained
- ReAct pattern (Reason + Act) explained
- LangGraph tutorial
- CrewAI tutorial
- AutoGen tutorial
- Multi-agent systems explained
- Tool calling / function calling for agents
- Agent memory (short-term vs long-term) explained
- Model Context Protocol (MCP) explained
- Building an MCP server tutorial
- Agent orchestration patterns
- Agentic workflows vs traditional automation

Checkpoint: Build one simple agent that uses at least 2 tools (e.g., a search tool + a database query tool) to complete a multi-step task.

Phase 5: MLOps / LLMOps & Deployment (1-2 weeks)

Where the real money is — keeping AI systems alive in production. You already know Docker/K8s basics from microservices, so this is mostly translation.

- What is LLMOps explained
- Prompt versioning and management
- LLM observability tools (LangSmith, Langfuse)
- Monitoring LLM applications in production
- Cost optimization for LLM applications
- Guardrails for LLM applications (safety, validation)
- Evaluating LLM outputs automatically
- CI/CD for AI applications
- Deploying Spring Boot AI apps on AWS/Azure/GCP
- Docker for AI applications

Checkpoint: Add basic logging/monitoring and cost tracking to one of your earlier projects.

Phase 6: Flagship Project + Portfolio (3-4 weeks)

This is what actually gets you hired — real, deployed projects beat certificates.

- Pick ONE of these (or your own variant):
 - RAG-based support/documentation chatbot for a real dataset
 - Multi-agent system that automates a real workflow (e.g., resume screening extending your earlier SmartHire idea)
 - AI-powered extension of your IMS capstone (e.g., a natural-language inventory query agent)
- System design for GenAI applications
- Writing a good README for AI/ML GitHub projects
- Deploying a full-stack AI app (frontend + Spring Boot + LLM + vector DB)
- Preparing GenAI project talking points for interviews

Checkpoint: One fully deployed, documented, publicly visible project you can walk an interviewer through end-to-end.

Suggested pace

- Total: ~12-16 weeks at a steady pace (faster if you go intensive on weekends)
- Don't skip building — each phase's checkpoint is non-negotiable, it's what turns "I read about it" into "I can build it"
- Revisit Phase 1-2 topics as needed; RAG concepts get referenced everywhere else

Notes

- Keep using Claude for concept-first explanations + targeted code snippets as you go, the same way you've been working through your capstone and expense tracker
- When you hit a phase involving Java integration, come back and we can debug/build together like we did with IMS